



AI DATA SCIENCE MULTI-AGENT PIPELINE

A portfolio-ready project walkthrough for recruiters, senior engineers and HR

Built with: Python • AutoGen • Gemini • Streamlit • Pandas • NumPy • Scikit-Learn

Developer: Ajim Mujawar

The project turns a raw CSV dataset into an AI-assisted data-science workflow using multiple specialized agents instead of one monolithic script.

LinkedIn Portfolio PDF • 2026

1. Project at a Glance

Goal: automate common data-science steps through a coordinated team of AI agents. A user uploads a CSV, chooses a target column, and the system coordinates analysis, cleaning, visualization, machine learning, evaluation and report generation.

Area	Implementation
Interface	Streamlit dashboard
Agent framework	Microsoft AutoGen / AgentChat
LLM	Gemini via API / model client
Data	CSV datasets using Pandas
ML	Scikit-Learn pipeline and saved model artifacts
Outputs	Charts, evaluation results, AI-generated report
Example dataset	Amazon laptop dataset

Why it is interesting

- Shows agent orchestration rather than only a single ML model.
- Separates responsibilities into reusable Python modules.
- Provides a visual Streamlit interface suitable for demonstrations.
- Produces both analytical artifacts and a human-readable AI report.
- Can be extended with new agents, models and data-processing steps.

2. End-to-End Multi-Agent Workflow

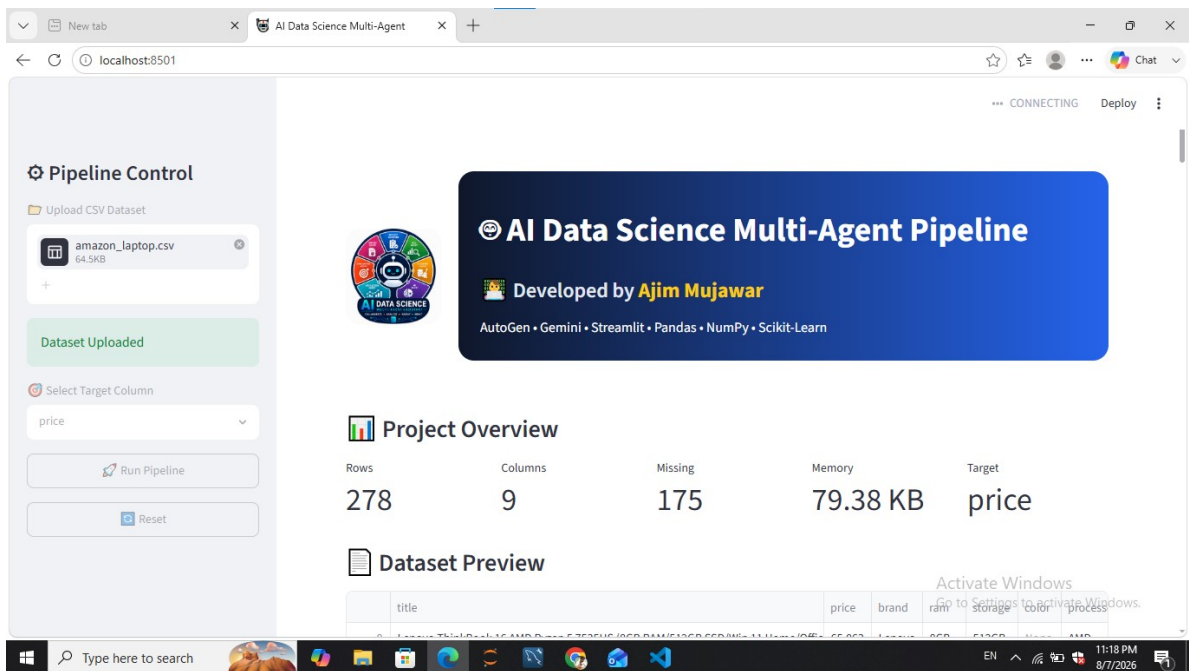
The recommended flow for a recruiter demo is: upload → inspect → orchestrate → analyze → clean → visualize → train/evaluate → report.



Key idea: the Manager Agent coordinates specialized agents. Each agent has a narrow responsibility, making the system easier to explain, test and extend.

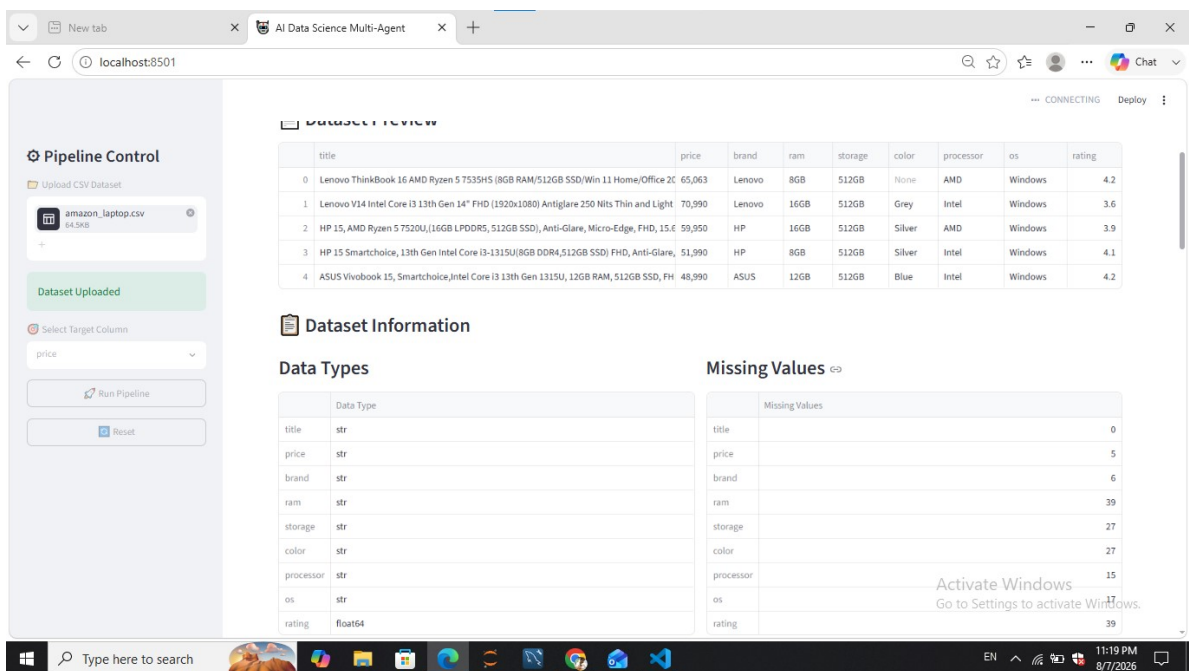
3. Product Walkthrough — Streamlit Dashboard

Step 1 — Project Overview



The dashboard introduces the application, shows dataset size and target selection, and gives the user a single place to start the pipeline.

Step 2 — Dataset Inspection



The system previews the uploaded dataset and surfaces data types and missing-value information before modeling.

Step 3 — Statistical Summary + Agent Status

The screenshot shows the AI Data Science Multi-Agent Pipeline dashboard. On the left is the Pipeline Control panel, which includes an 'Upload CSV Dataset' section with a file named 'amazon_laptop.csv' (4.5 KB) and a 'Dataset Uploaded' status. Below this is a 'Select Target Column' dropdown set to 'price', and buttons for 'Run Pipeline' and 'Reset'. The main area is titled 'Statistical Summary' and contains a table with the following data:

	title	price	brand	ram	storage	color	processor	os	rating
count	278	273	272	239	251	251	263	261	239
unique	278	213	17	12	6	9	5	3	None
top	Lenovo ThinkBook	68,990	HP	16GB	512GB	Silver	Intel	Windows	None
freq	1	6	94	110	167	98	176	256	None
mean	None	None	None	None	None	None	None	None	3.8883
std	None	None	None	None	None	None	None	None	0.7526
min	None	None	None	None	None	None	None	None	1
25%	None	None	None	None	None	None	None	None	3.6
50%	None	None	None	None	None	None	None	None	4
75%	None	None	None	None	None	None	None	None	4.2

Below the table is the 'AI Agents Dashboard' showing the status of several agents: Manager Agent (Running), Visualization Agent (Waiting), EDA Agent (Waiting), and Evaluation Agent (Waiting). A Windows watermark is visible on the right side of the dashboard.

The dashboard combines statistical summary information with the status of the multi-agent team.

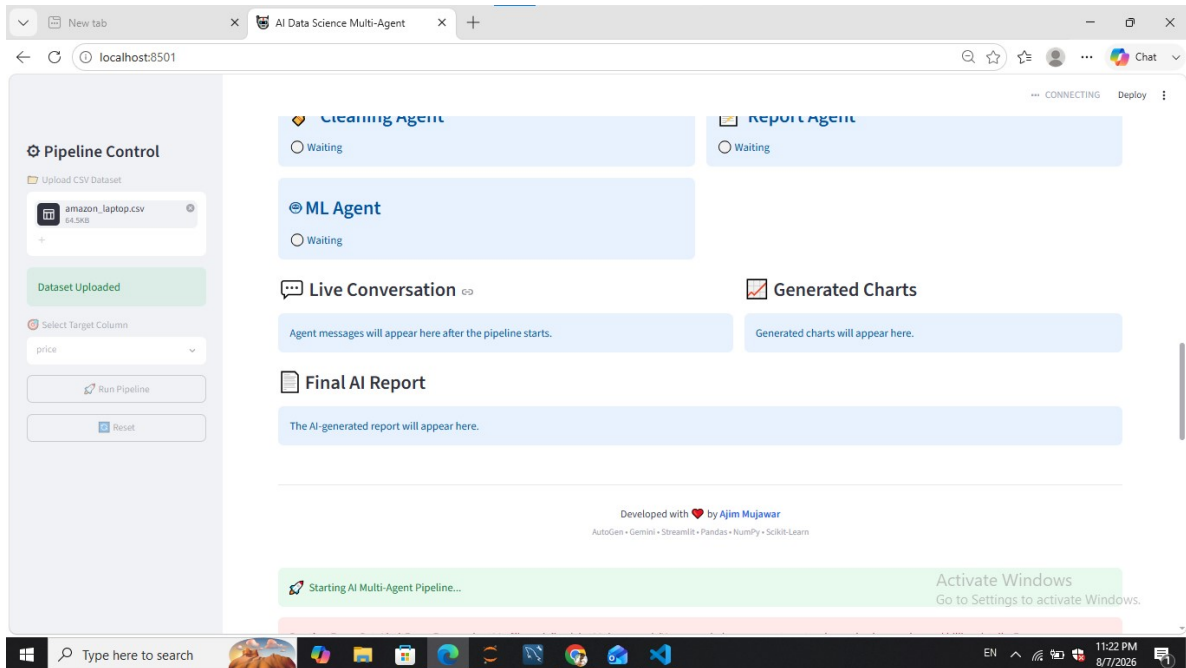
Step 4 — Agent Team

The screenshot shows the AI Data Science Multi-Agent Pipeline dashboard, similar to the previous one but with a different layout for the AI Agents Dashboard. The Pipeline Control panel on the left remains the same. The 'AI Agents Dashboard' now displays seven specialized roles: Manager Agent (Running), Visualization Agent (Waiting), EDA Agent (Waiting), Evaluation Agent (Waiting), Cleaning Agent (Waiting), Report Agent (Waiting), and ML Agent (Waiting). Below these agents are sections for 'Live Conversation' (with a placeholder message) and 'Generated Charts' (with a placeholder message). A Windows watermark is also present.

Seven specialized roles are visible: Manager, EDA, Cleaning, Visualization, Evaluation, ML and Report.

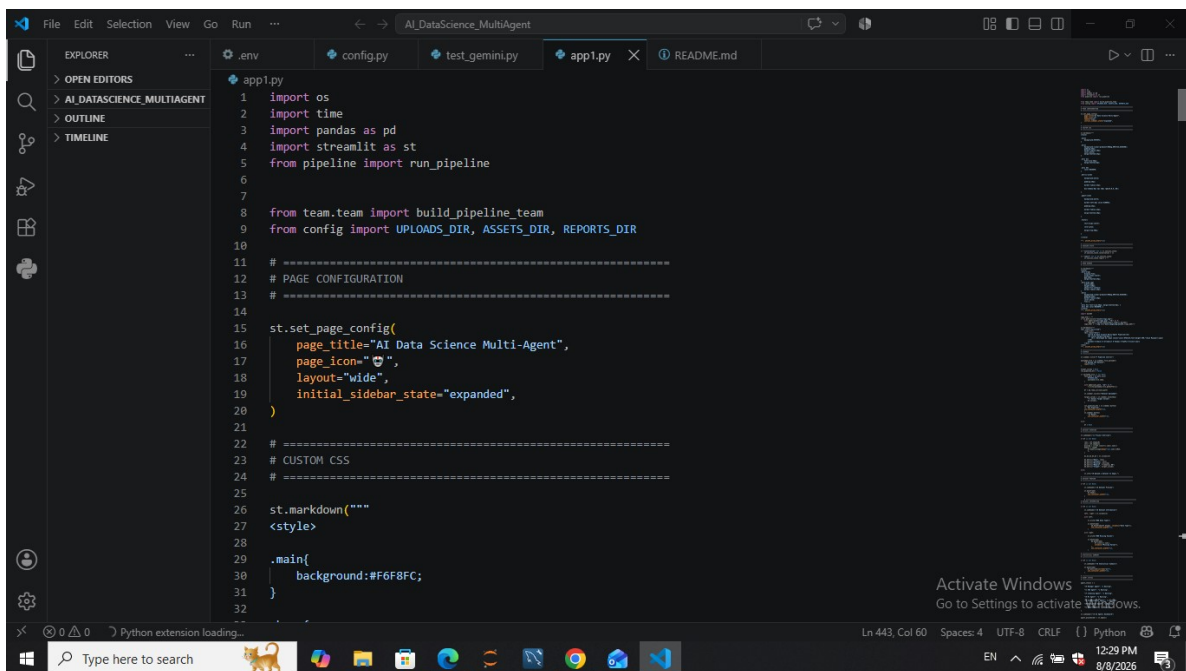
4. Output & Reporting

Step 5 — Charts, Conversation and Final AI Report



The intended final stage is to show generated charts, live agent messages and a consolidated AI-generated report.

Step 6 — Application Code



The Streamlit entry point connects the user interface with the pipeline and configuration modules.

5. Project Structure

The folder structure follows a modular design: agents contain agent roles, components contain UI pieces, team contains orchestration, utils contain reusable helpers, and reports/models hold outputs.

```
AI_DataScience_MultiAgent/  
  .env  
  app1.py  
  config.py  
  model_client.py  
  pipeline.py  
  project_structure.txt  
  README.md  
  requirements.txt  
  test_chat.py  
  test_gemini.py  
  test_models.py  
  test_visualization.py  
    
  agents/  
    cleaning_agent.py  
    eda_agent.py  
    evaluation_agent.py  
    manager.py  
    ml_agent.py  
    report_agent.py  
    visualization_agent.py  
    
  components/  
    charts.py  
    dataframe_view.py  
    sidebar.py  
    upload_component.py  
    
  data/  
  logs/  
  models/  
    best_model.joblib  
  notebooks/  
  reports/  
    model_meta.json  
    report.md  
    report.txt  
  team/  
    team.py  
  uploads/  
    amazon_laptop.csv  
    amazon_laptops.csv  
    avocado.csv  
    cleaned_data.csv  
  utils/  
    file_handler.py  
    helper.py  
    plotting.py  
    preprocessing.py  
    report_generator.py
```

Engineering signal for recruiters: this is not only a notebook. The codebase is organized as an application with configuration, agents, UI components, orchestration, tests, models and reports.

6. README-Style Project Summary

Project: AI Data Science Multi-Agent Pipeline

Description: A Streamlit-based AI data-science assistant that coordinates multiple specialized AutoGen agents to inspect data, clean it, visualize patterns, train/evaluate models and produce a final report.

Core modules

Agent	Responsibility
Manager Agent	Coordinates the overall workflow and delegates work.
EDA Agent	Profiles columns, distributions, missing values and statistical patterns.
Cleaning Agent	Handles preprocessing/cleaning tasks required before modeling.
Visualization Agent	Creates charts and visual summaries.
ML Agent	Runs machine-learning tasks and stores model artifacts.
Evaluation Agent	Checks model performance and validation results.
Report Agent	Combines findings into a final human-readable report.

Run concept

1. Create environment and install requirements
2. Configure API key in .env (never commit the key)
3. Start Streamlit
4. Upload CSV
5. Select target column
6. Run pipeline
7. Review agent messages, charts, model results and report

Suggested command: streamlit run app1.py

7. Beginner Multi-Agent Planner — Simple AutoGen Example

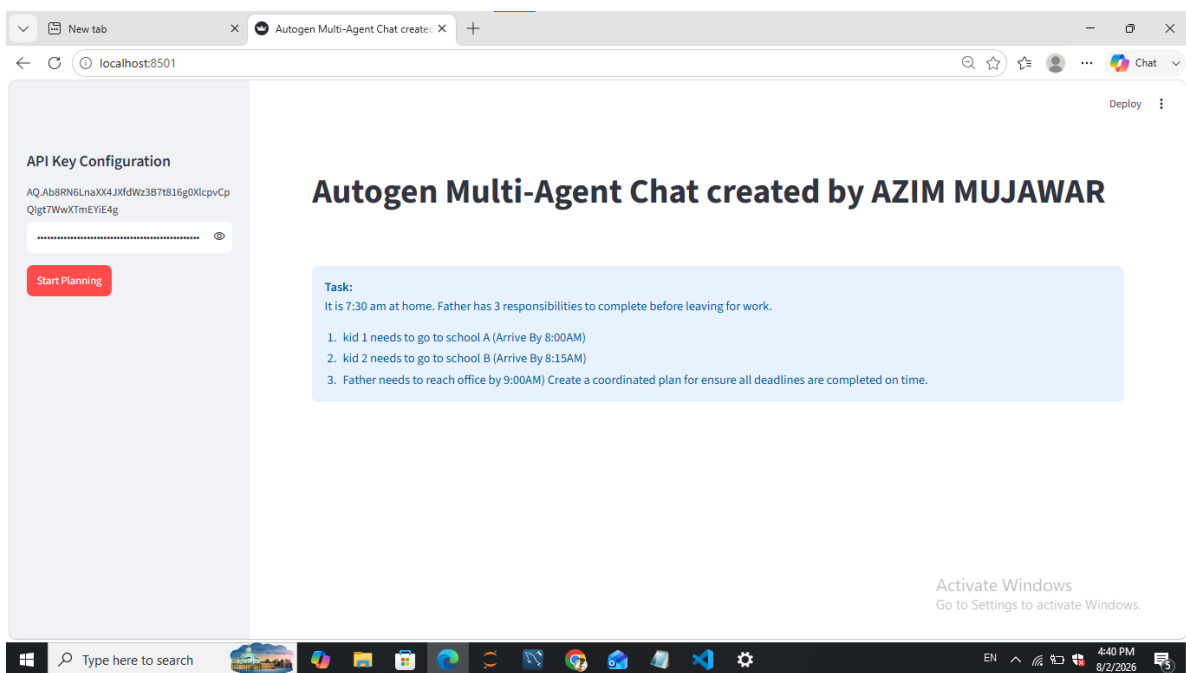
A second, simpler project is useful for learning the core idea before working with the larger data-science pipeline. The example shown in the screenshots is a family schedule planner: separate agents handle Kid 1, Kid 2 and the overall coordination problem.

Beginner agent design

Agent	Simple responsibility
School Agent 1	Plan Kid 1's trip to School A before 8:00 AM.
School Agent 2	Plan Kid 2's trip to School B before 8:15 AM.
Manager / Coordinator	Combine the individual plans into one feasible schedule.

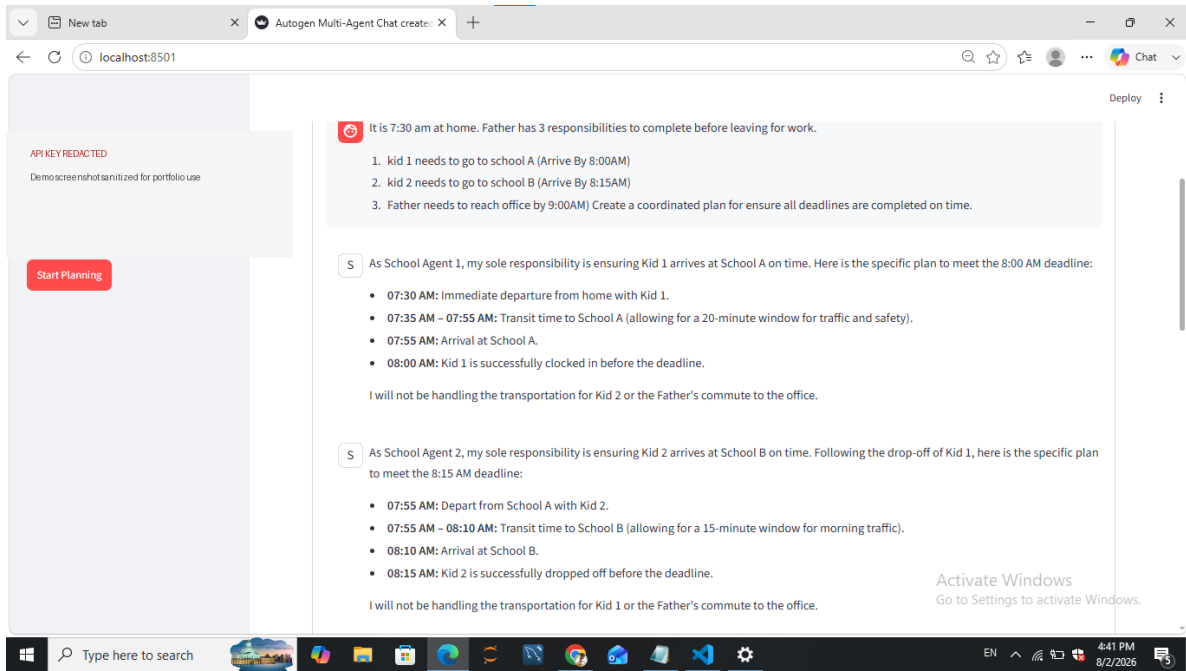
Learning progression: **one agent** → **two specialist agents** → **coordinator** → **group conversation** → **real-world data-science agents**.

Beginner Planner — Step 1: Define the Task



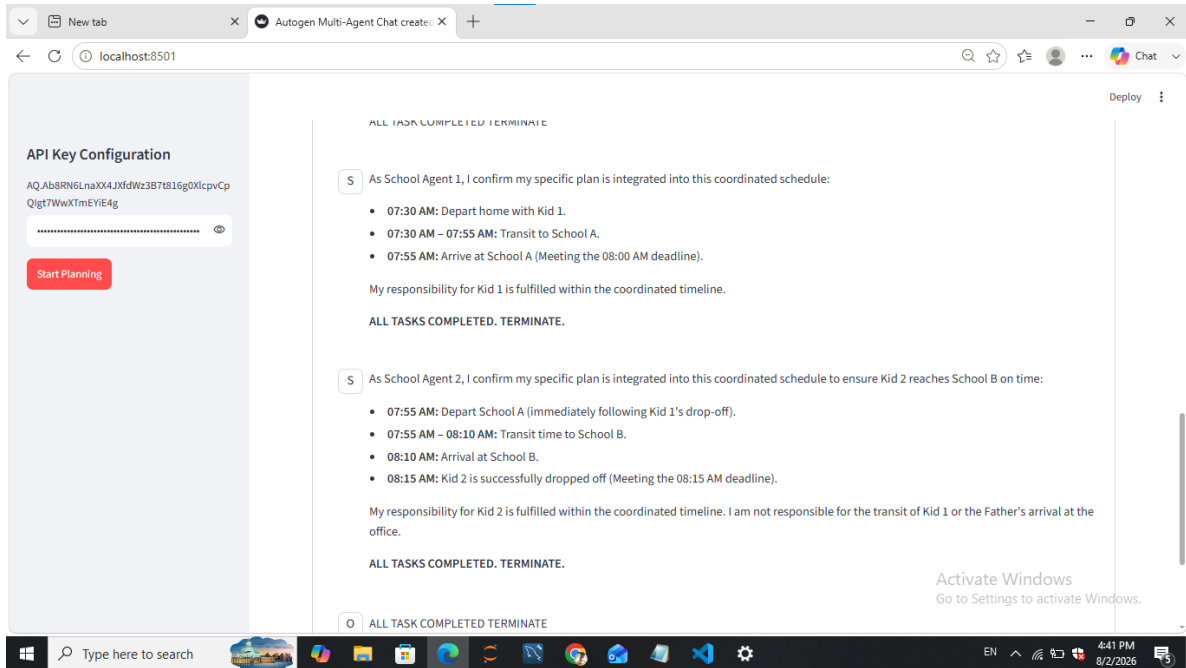
The user gives one scheduling problem with multiple deadlines. This is a clean beginner example because the responsibilities are easy to understand.

Beginner Planner — Step 2: Specialist Agents Respond



Two school agents produce separate time-based plans. The API key area has been intentionally redacted for portfolio safety.

Beginner Planner — Step 3: Coordinated Completion



The conversation shows agents confirming their responsibilities and terminating after the schedule is completed.

Why this beginner project matters

It demonstrates the fundamental Agentic AI pattern without complicated ML code: each agent has a role, the agents communicate, and the combined result solves a larger task. This makes it a strong stepping stone toward the full AI Data Science Multi-Agent Pipeline.

8. Technical Highlights for HR & Senior Review

Agentic architecture: Specialized agents collaborate instead of putting every responsibility into one prompt.

Modular Python design: Agent, UI, utility, team and pipeline responsibilities are separated into modules.

Data-science workflow: The system covers inspection, cleaning, visualization, ML, evaluation and reporting.

Application mindset: Streamlit provides an interactive product-like interface rather than only notebook output.

Artifact handling: Models and reports are persisted in dedicated folders for repeatable workflows.

Extensibility: New agents can be added without redesigning the entire application.

9. Important Engineering Notes

API quota: A Gemini 429 RESOURCE_EXHAUSTED error can occur when the free-tier request quota is exceeded. This is an API usage/plan limitation, not automatically a failure of the Python orchestration code. For a live demo, keep prompts short, reduce unnecessary model calls, or use an available model/project quota.

Security: Never publish API keys in screenshots, GitHub, README files or LinkedIn posts. The original planner screenshot contained a visible key, so the version in this portfolio has been redacted. If the displayed key was real, rotate/revoke it before publishing.

10. Recommended LinkedIn Story

Hook: "What if a data-science workflow could behave like a team instead of a single script?"

Then show the sequence: **Dashboard** → **Agent Team** → **Data Inspection** → **EDA/Cleaning** → **ML/Evaluation** → **Charts/Report** → **Beginner Planner**.

Suggested hashtags: #AgenticAI #MultiAgentSystems #AutoGen #GenerativeAI #DataScience #MachineLearning #Python #Streamlit #AIEngineering

11. Portfolio Takeaway

This project demonstrates a progression from a beginner multi-agent planner to a larger AI-assisted data-science application. The strongest portfolio message is not simply “I used Gemini”; it is that the project separates responsibilities, coordinates agents, handles data workflows and presents results through an application interface.

Recommended demo order:

1. Upload a CSV and select a target.
2. Show the project overview and dataset preview.
3. Show the agent dashboard and explain each role.
4. Run the pipeline and capture agent activity.
5. Show generated charts/model results/report.
6. Finish with the beginner planner to explain the Agentic AI concept simply.

AI Data Science Multi-Agent Pipeline
Developed by Ajim Mujawar

Portfolio document prepared from the supplied project screenshots and folder structure.